Operating Systems Lab

Week 6: Banker's Algorithm

1. Safety Algorithm

Code:

```c
#include <stdio.h>

int prs; // number of processes
int rcs; // number of resources

void safety(int alloc[prs][rcs], int max[prs][rcs], int avail[rcs])
{
    int need[prs][rcs];
    int work[rcs];
    int finish[prs];
    int seq[prs];
    int done = 0;

    // init need = max - alloc
    for (int i = 0; i < prs; i++)
        for (int j = 0; j < rcs; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    // init work = available
    for (int i = 0; i < rcs; i++) // [i0 i1 i2 ...]
        work[i] = avail[i];

    // init finish[i] = false for all i
    for (int i = 0; i < prs; i++)
        finish[i] = 0;

    while (done < prs)
    {
        // finding suitable i
        int found = 0;
        for (int i = 0; i < prs; i++)
        {
            if (!finish[i]) // check finish[i] = false
            {
                int check = 1;
```

```
                for (int j = 0; j < rcs; j++)
                {
                    if (need[i][j] > work[j]) // check need_i <= work
                    {
                        check = 0;
                        break;
                    }
                }

                if (check)
                {
                    for (int k = 0; k < rcs; k++)
                        work[k] += alloc[i][k];

                    finish[i] = 1;
                    seq[done++] = i;
                    found = 1;
                }
            }
        }
        if (!found)
            break;
    }
    if (done < prs)
        printf("\nSystem is in an unsafe state.\n");

    else
    {
        printf("\nSystem is in a safe state.\n");
        printf("Safe sequence: ");
        for (int i = 0; i < done; i++)
            printf("P%d ", seq[i]);
    }
}

int main()
{
    printf("Safety Algorithm\n");
    printf("Royden Miranda 1WA24CS240\n");

    printf("Enter the number of processes: ");
    scanf("%d", &prs);

    printf("Enter the number of resources: ");
    scanf("%d", &rcs);
```

```
    // m x n (prs x rcs) matrices
    int alloc[prs][rcs];
    int max[prs][rcs];
    int avail[rcs];

    printf("\nEnter Allocation: \n");
    for (int i = 0; i < prs; i++)
        for (int j = 0; j < rcs; j++)
            scanf("%d", &alloc[i][j]);

    printf("\nEnter Max Demand: \n");
    for (int i = 0; i < prs; i++)
        for (int j = 0; j < rcs; j++)
            scanf("%d", &max[i][j]);

    printf("\nEnter Available Resources: \n");
    for (int i = 0; i < rcs; i++)
        scanf("%d", &avail[i]);

    safety(alloc, max, avail);

    return 0;
}
```

Output:

```
Safety Algorithm
Royden Miranda 1WA24CS240

Enter the number of processes: 5
Enter the number of resources: 3

Enter Allocation:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Demand:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence: P1 P3 P4 P0 P2
```